

Систематический алгоритм контекстного truecasing для BookCorpus

Цель: сделать анализ не выборочным, а обязательным многошаговым процессом с контролем покрытия. Основной интеллектуальный truecasing выполняет LLM по контексту; Python используется только как детерминированный исполнитель решений и для проверки инвариантов.

1. Разделение на книги и фрагменты

Сначала определяются границы книг/фрагментов. Контекст сущностей действует только внутри текущей книги. При новой книге накопленный реестр сущностей сбрасывается.

2. Проход по тексту блоками

Текст просматривается последовательно блоками (например, 50-100 строк), но каждое семантическое решение принимается по локальному окну примерно 5-8 предложений. При недостатке контекста окно расширяется.

3. Фиксированный набор категорий для проверки

В каждом блоке LLM обязана проверять не только основные имена персонажей, а все элементы, которые могут требовать регистра, отличного от lowercase: PERSON, SURNAME, PLACE, ORG, PRODUCT/BRAND, WORK/TITLE, ABBREVIATION, религиозные/исторические имена и другие proper nouns.

4. Накопительный реестр сущностей

```
surface: laney
canonical: Laney
type: PERSON
scope: book_2
confidence: high
```

Для однозначных сущностей реестр может использоваться для распространения правильного регистра по текущей книге.

5. Второй проход: аудит пропущенных сущностей

После первичного entity pass выполняется повторный проход по всей книге. Его задача - найти lowercase-слова и фразы, которые по контексту ведут себя как имена/названия, но отсутствуют в entity map.

Контрольный вопрос для каждого блока: «Есть ли здесь lowercase-слово, которое по контексту ведёт себя как proper noun, но ещё не зарегистрировано?»

6. Отдельный аудит редких сущностей

Особое внимание уделяется сущностям, встречающимся 1-2 раза. Python может организационно собрать редкие токены/фразы и показать их контексты, но не принимает семантическое решение.

```
marlowe - 1 occurrence

context:
she handed the papers to marlowe .
"tell him i'll call tomorrow."
```

```
LLM decision:
marlowe -> Marlowe
```

7. Неоднозначные слова обрабатываются по употреблению

Нельзя слепо глобально капитализировать неоднозначные слова вроде will, rose, summer, hunter, grey, king, chance, faith, hope. Для них решение принимается отдельно для каждого occurrence.

```
line 431: rose = PERSON -> Rose
line 912: rose = VERB -> rose
line 1402: rose = PERSON -> Rose
```

8. Глобальное распространение только для однозначных сущностей

Однозначные сущности, подтверждённые контекстом, можно распространять по книге. Неоднозначные слова меняются адресно, только в тех местах, где контекст подтверждает proper noun.

9. Три обязательных LLM-прохода

```
PASS 1
структура книги + основные сущности
|
PASS 2
поиск пропущенных и редких сущностей
|
PASS 3
проверка неоднозначных употреблений
|
финальная entity/occurrence map
|
Python применяет case
```

10. Финальный контроль покрытия

После формирования результата выполняется дополнительная проверка слов, которые остались lowercase во всех употреблениях, но встречаются в позициях, характерных для имён/названий. Они повторно показываются LLM с контекстом и либо капитализируются, либо явно отвергаются.

```
unknown candidates:
marlowe
karesh
eldon
tumblin
```

Желаемое свойство процесса: каждый потенциальный кандидат либо капитализирован, либо был явно рассмотрен и оставлен lowercase.

11. Роль Python

Python не выполняет NER и не решает, является ли слово именем. Он только применяет решения LLM, поддерживает реестр/occurrence map и проверяет, что никаких изменений кроме регистра не произошло.

```
assert original.lower() == converted.lower()
assert original_id == converted_id
assert original_keys == converted_keys
assert original_line_count == converted_line_count
```

Рабочая схема для следующих файлов

1. Определить границы книг.
2. Primary entity pass.
3. Rare/missed-entity audit.
4. Ambiguous-occurrence audit.
5. Сформировать entity map + occurrence map.
6. Применить решения Python.
7. Проверить invariants.
8. Вернуть *_truecased.jsonl.